

Manual Técnico Gestor Automatizado de Servicios con Notificación por Telegram

Proyecto Integrador 2026

Ingeniería en Ciberseguridad e Infraestructura de Cómputo

Este documento describe la instalación, configuración, arquitectura, operación y pruebas del sistema automatizado desarrollado en Bash para administrar Zorin OS Pro y sistemas GNU/Linux compatibles.

1. Objetivo y alcance

El sistema integra siete scripts independientes que comparten un archivo de configuración. Sus funciones cubren gestión de usuarios, respaldos, monitoreo de recursos, supervisión de servicios, ejecución remota, monitoreo de red e inventario del sistema. Los eventos relevantes se registran en archivos de log y pueden notificarse a un bot de Telegram.

Cada script incluye shebang de Bash, validación de entradas, funciones modulares, códigos de salida y captura de las señales SIGINT y SIGTERM.

2. Requisitos

Zorin OS Pro con Bash 4 o superior. Las herramientas principales son awk, sed, tar, find, df, free, ps, curl, ping, ssh, scp y timeout. Para revisar puertos se utiliza nc, nmap o /dev/tcp según disponibilidad.

En Zorin OS Pro se pueden instalar las dependencias con:

```
sudo apt update
sudo apt install bash curl openssh-client tar gzip bzip2 xz-utils iputils-ping netcat-openbsd
```

La administración de usuarios y el reinicio de servicios requieren permisos de administrador. Los demás módulos deben ejecutarse con un usuario regular.

3. Estructura del proyecto

config.txt	Configuración y funciones compartidas
setup.sh	Instalación en /opt/sistema-servicios
configurar-telegram.sh	Configuración externa y validación del bot
test.sh	Pruebas locales sin efectos administrativos
usuarios.sh	Alta, baja, modificación y consulta de usuarios
respaldo.sh	Creación y verificación de respaldos
monitoreo.sh	Lectura de CPU, memoria y disco
servicios.sh	Supervisión y reinicio de servicios
remoto.sh	Copia y ejecución mediante SSH
red.sh	Verificación de ping y puertos
inventario.sh	Reporte de hardware y software
hosts.txt	Lista de hosts remotos
crontab.txt	Ejemplos de programación periódica

4. Instalación

Desde la raíz del repositorio se debe otorgar permiso de ejecución y lanzar el instalador como administrador:

```
chmod +x ./*.sh
sudo ./setup.sh
```

El modo --check valida Zorin OS y las dependencias sin realizar cambios. El instalador copia los scripts a /opt/sistema-servicios, crea los directorios /var/log/sistema-servicios, /var/log/reportes y /var/backups/sistema-servicios, configura permisos y crea el usuario backup si no existe.

Después de instalar, se debe editar:

```
sudo nano /opt/sistema-servicios/config.txt
```

5. Configuración compartida

Todos los módulos cargan config.txt desde el mismo directorio del script. Los parámetros principales son:

LOG_DIR	Directorio de logs
BACKUP_DIR	Destino de respaldos
REPORTS_DIR	Reportes de ejecución remota
INVENTORY_DIR	Destino de inventarios, por defecto /var/log
UMBRAL_CPU	Umbral de CPU, por defecto 70
UMBRAL_DISCO	Umbral de disco, por defecto 70
UMBRAL_MEMORIA	Umbral de memoria
SERVICIOS	Servicios supervisados
HOSTS_PING	Hosts revisados por red.sh
PUERTOS_CRITICOS	Puertos que deben responder
DIRS_RESPALDO	Directorios incluidos en respaldos
RESPALDO_COMPRESION	gzip, bzip2 o xz
RESPALDO_RETENCION	Días de conservación
USUARIO_REMOTO	Usuario SSH predeterminado
TIMEOUT_SSH	Tiempo máximo de operación remota

Telegram permanece deshabilitado hasta configurarlo. Las credenciales se guardan fuera del repositorio en /etc/sistema-servicios/telegram.env con permisos restrictivos:

```
sudo /opt/sistema-servicios/configurar-telegram.sh
```

El configurador oculta el token durante la captura, valida el bot mediante la API y envía un mensaje de prueba. Las variables de entorno siguen disponibles para ejecuciones temporales.

6. usuarios.sh

Muestra un menú interactivo para crear, eliminar, modificar y listar usuarios. Antes de cada acción valida la existencia del usuario. La creación valida el shell solicitado y asigna la contraseña mediante chpasswd. Cada operación se registra y genera una notificación de Telegram.

El nombre debe usar minúsculas, números, guion o guion bajo. La contraseña requiere al menos ocho caracteres y confirmación antes de crear la cuenta. Si la asignación falla, el usuario se elimina automáticamente para revertir la operación incompleta.

```
sudo ./usuarios.sh
```

Este módulo requiere root porque utiliza useradd, usermod, userdel y chpasswd.

7. respaldo.sh

Comprime uno o varios directorios con tar, comprueba que el archivo exista, valida que su tamaño sea mayor que cero y revisa la integridad del contenedor. Después elimina respaldos que excedan el período de retención con-

figurado.

La notificación contiene nombre, ruta, tamaño y fecha del respaldo. Para cron se recomienda ejecutar con el usuario backup y permisos únicamente sobre los directorios necesarios.

8. monitoreo.sh

Obtiene el porcentaje de CPU, disco y memoria. Registra todas las lecturas y compara los valores con umbrales configurables. Si alguno se supera, envía una alerta de Telegram.

9. servicios.sh

Lee la lista de servicios desde config.txt o desde los argumentos. Comprueba el estado con systemctl y, como alternativa, service. Si detecta un servicio inactivo intenta reiniciarlo, vuelve a verificarlo, registra el resultado y notifica tanto el éxito como el fallo.

```
sudo ./servicios.sh
sudo ./servicios.sh nginx ssh
```

10. remoto.sh

Lee hosts desde un archivo externo. Cada línea puede contener un hostname, una IP o el formato usuario@host. Copia el script con scp, lo ejecuta mediante SSH, captura su salida, elimina la copia temporal y genera un reporte individual con timestamp y código de salida.

El módulo exige autenticación por llaves para funcionar sin interacción y ser compatible con cron. diagnostico.sh permite demostrar la ejecución sin depender de config.txt en el host remoto. Al finalizar se envía a Telegram un resumen de hosts exitosos, fallidos y la ruta del reporte.

11. red.sh

Lee hosts desde config.txt o un archivo, valida los puertos solicitados y comprueba ping y conectividad TCP. Cada host se clasifica como accesible, parcialmente accesible o sin respuesta. Los hosts caídos y los puertos críticos cerrados se incluyen en el log y en la alerta de Telegram.

12. inventario.sh

Recopila modelo y núcleos de CPU, memoria total y disponible, almacenamiento por partición, sistema operativo, kernel, hostname, interfaces, servicios y usuarios. El reporte se guarda como /var/log/inventario_FECHA.txt y se envía un resumen por Telegram.

13. Logs y reportes

Los eventos de cada módulo se almacenan en LOG_DIR con el formato fecha, hora y descripción. remoto.sh utiliza REPORTS_DIR para sus reportes por host. inventario.sh utiliza INVENTORY_DIR y respaldo.sh utiliza BACKUP_DIR.

Los directorios pueden sustituirse mediante variables de entorno para pruebas o instalaciones sin acceso a /var.

14. Programación con cron

El archivo crontab.txt contiene ejemplos completos. Algunas tareas recomendadas son:

```
0 2 * * * /opt/sistema-servicios/respaldo.sh
*/5 * * * * /opt/sistema-servicios/monitoreo.sh
*/10 * * * * /opt/sistema-servicios/servicios.sh
*/15 * * * * /opt/sistema-servicios/red.sh
0 1 * * 1 /opt/sistema-servicios/inventario.sh
```

Antes de instalar una tarea se deben comprobar rutas, permisos y variables de entorno. Los tokens de Telegram pueden cargarse desde un archivo protegido que no pertenezca al repositorio.

15. Casos de prueba

15.1. Prueba automatizada segura

El comando `./test.sh` crea un entorno temporal, valida la sintaxis de todos los scripts y ejecuta monitoreo, inventario y respaldo. No crea usuarios, no reinicia servicios y no establece conexiones SSH.

15.2. Prueba de monitoreo

Ejecutar `./monitoreo.sh -c 1 -d 1 -m 1` en un entorno de demostración. Se espera estado de alerta, registro de la lectura y notificación si Telegram está activo.

15.3. Prueba de respaldo

Crear un directorio temporal con un archivo y ejecutar `respaldo.sh` sobre él. Se espera un archivo comprimido válido, tamaño mayor que cero, log y resumen.

15.4. Prueba de servicios

Utilizar un servicio de laboratorio que pueda detenerse y reiniciarse sin afectar al equipo. Se espera detección, intento de reinicio y segunda revisión.

15.5. Prueba remota

Configurar un host de laboratorio con llave SSH y ejecutar un script inocuo, por ejemplo `hostname`. Se espera copia, salida remota y reporte individual.

15.6. Prueba de red

Usar localhost con un puerto abierto y otro cerrado. Se espera clasificación parcial, detalle del puerto cerrado y alerta cuando Telegram esté habilitado.

16. Seguridad y mantenimiento

Los scripts deben ejecutarse con el mínimo privilegio necesario. `config.txt` debe tener permisos restrictivos si contiene datos sensibles. SSH debe utilizar llaves, usuarios limitados y verificación de hosts en ambientes reales. Los logs y respaldos requieren políticas de retención y revisión periódica.

El token que haya sido publicado alguna vez debe revocarse y reemplazarse desde BotFather. El repositorio no debe contener tokens, contraseñas ni identificadores privados.

17. Guía de demostración final

La presentación puede seguir este orden: explicar `config.txt`; ejecutar `test.sh`; mostrar monitoreo con umbrales bajos; crear un respaldo de prueba; generar el inventario; revisar un host y sus puertos; mostrar un reporte remoto; y cerrar con logs y notificaciones de Telegram. Cada integrante debe explicar las funciones, validaciones y códigos de salida del módulo que presenta.

18. Conclusión

El proyecto cumple la arquitectura solicitada de siete scripts autónomos con configuración compartida, registro de eventos, validación, manejo de señales y notificaciones. La operación real depende de configurar correctamente permisos, servicios, hosts SSH y credenciales externas de Telegram.